

Systeme Linux

Programmation Shell

- ❖ Introduction aux scripts Shell
- ❖ Les variables
- ❖ Les conditions
- ❖ Les boucles
- ❖ Les fonctions

Introduction aux scripts shell

- ❖ Toutes les commandes que nous nous avons vu jusqu'à présent sont standards et ne sont pas propres à un shell particulier.
- ❖ Avant d'entamer la programmation shell elle-même, il nous faut voir quelques options détaillées du shell par défaut, le Bourne Shell
- ❖ L'interprète de commandes (shell) permet d'interagir avec le système
- ❖ Exécution de commandes (modification / consultation de l'état du système)
- ❖ Utilisation avancée : combinaison de commandes par exemple en redirigeant les entrées/sorties
- ❖ Programmation shell : combinaison de commandes au sein d'un script dans le but d'automatiser certaines tâches

Introduction aux scripts shell

- ❖ Le shell est le programme qui gère l'invite de commandes. C'est donc le programme qui attend que vous rentriez des commandes:
 - Auto compléter une commande ou un nom de fichier
 - Gérer les processus (envoi en arrière-plan, mise en pause avec Ctrl + Z...)
 - Rediriger et chaîner les commandes (les fameux symboles >, <, |, etc.)
- ❖ Le shell fournit toutes les fonctionnalités de base pour pouvoir lancer des commandes

Gestion des processus et des flux de données

Introduction aux scripts shell

- ❖ Une commande peut être interne ou externe.
- ❖ Une commande interne est une commande directement incluse et interprétée par le shell, sans passer par un quelconque exécutable. Un exemple est la commande `cd`.
- ❖ Une commande externe est un exécutable que le shell recherchera dans une liste de chemins prédéfinis, le `PATH`, puis exécutera en tant que processus `ls`.

Introduction aux scripts shell

❖ Les principaux shell existants

sh : *Bourne Shell*. L'ancêtre de tous les shells.

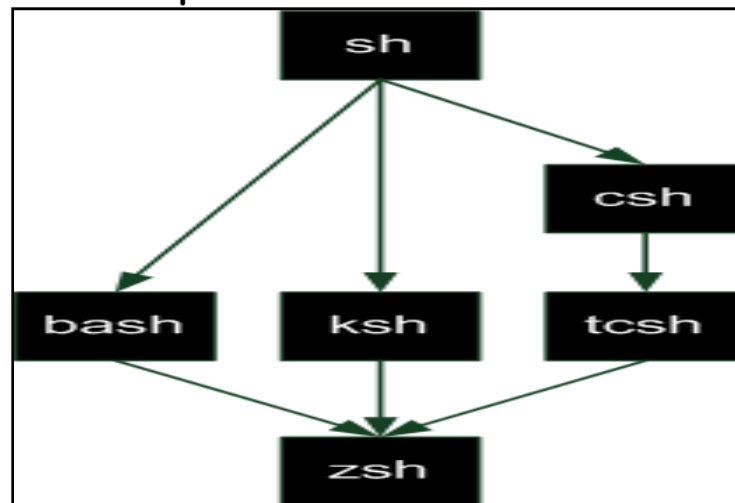
bash : *Bourne Again Shell*. Une amélioration du *Bourne Shell*, disponible par défaut sous Linux et Mac OS X.

ksh : *Korn Shell*. Un shell puissant assez présent sur les Unix propriétaires, mais aussi disponible en version libre, compatible avec bash.

csh : *C Shell*. Un shell utilisant une syntaxe proche du langage C.

tcsh : *Tenex C Shell*. Amélioration du *C Shell*.

zsh : *Z Shell*. Shell assez récent reprenant les meilleures idées de bash, ksh et tcsh.



Introduction aux scripts shell

- ❖ Le bash est installé sur Ubuntu par défaut.
- ❖ Si vous voulez essayer un autre shell, comme ksh par exemple, vous pouvez le télécharger comme n'importe quel paquet (apt-get install ksh, chsh, bin/ksh...
- ❖ Pour écrire du script shell il faut créer un fichier en utilisant un éditeur de texte (nano, vi...)
- ❖ Un Script shell correspond à un fichier exécutable d'extension .sh et débutant par : `#!/bin/bash`

Introduction aux scripts shell

- ❖ Un Script shell correspond à un fichier exécutable d'extension .sh.
- Première ligne : `#!/bin/bash`
- Ecrire les commandes que vous souhaitez exécuter
- les commentaires commencent par un #.
- Enregistrer le fichier et fermer l'éditeur
- donner le droit d'exécution au fichier
- Exécution du script en tapant « ./ » devant le nom du fichier (./script.sh)

Introduction aux scripts shell

- ❖ **Exécution de débogage:** `bash -x fichier.sh`
- ❖ Le `PATH` est une variable système qui indique où sont les programmes exécutables sur votre ordinateur. Si vous tapez `echo $PATH` vous aurez la liste de ces répertoires « spéciaux ».
- ❖ Copier le script dans un de ces répertoires comme `/bin`, `/usr/bin` ou `/usr/local/bin`.

Les Variables

Déclaration et affectation

- ❖ Une variable est une donnée identifiée par un nom. Le nom d'une variable est une suite de caractères alphanumérique.
- ❖ Un nom de variable obéit à certaines règles :
 - Il peut être composé de lettres minuscules, majuscules, de chiffres, de caractères de soulignement.
 - Le premier caractère ne peut pas être un chiffre
 - La taille d'un nom est en principe illimite (il ne faut pas abuser non plus)
 - Les conventions veulent que les variables utilisateur soient en minuscules pour les différencier des variables système. Au choix de l'utilisateur.

Les Variables

- ❖ Une variable est déclarée dès qu'une valeur lui est affectée. L'affectation est effectuée avec le signe =, sans espace avant ou après le signe.
exemple: var=Bonjour.
- ❖ la commande **echo** affiche dans la console tous les paramètres qu'elle reçoit
 - `echo -e` : Activer le retour à la ligne \n.
 - Pour afficher une variable, utiliser le nom précédé du symbole dollar \$
 - Pour afficher à la fois du texte et la variable, faire attention au « quotes ».

Exemple:

```
test='hello world'
```

```
echo $test
```

```
Echo 'le message est $test'  erreur
```

```
Echo "le message est $test"
```

Les Variables

read : Demander une saisie

Exemple:

- `read nom`
`echo "Bonjour $nom !"`
- `read -p 'Entrez votre nom : ' nom`
`echo "Bonjour $nom !"`
- `read -n` : limiter le nombre de caractères
- `read -t` : limiter le temps autorisé pour saisir un message
- `read -s` : ne pas afficher le texte saisi.

Les Variables

Les opérations arithmétiques

❖ En bash, les variables sont toutes des chaînes de caractères. En soi, le bash n'est pas vraiment capable de manipuler des nombres ; il n'est donc pas capable d'effectuer des opérations.

❖ Heureusement, il est possible de passer par des commandes). Ici, la commande à connaître est **let** . (+; - ^; *; **; /; %)

Exemple:

```
#!/bin/bash
```

```
let "a = 5"
```

```
let "b = 2"
```

```
let "c = a + b"
```

```
echo $c
```

Les Variables

Les variables d'environnement

❖ Les variables d'environnement sont des variables que l'on peut utiliser dans n'importe quel programme. On parle aussi parfois de **variables globales**. Vous pouvez afficher toutes celles que vous avez actuellement en mémoire avec la commande **env** :

Exemples:

- SHELL : indique quel type de shell est en cours d'utilisation (sh, bash...)
- PATH : une liste des répertoires qui contiennent des exécutables
- EDITOR : l'éditeur de texte par défaut
- HOME : la position de votre dossier home
- PWD : le dossier dans lequel vous vous trouvez
- OLDPWD : le dossier dans lequel vous vous trouviez auparavant.

❖ les noms de ces variables sont, par convention, écrits en majuscules.

Les Variables

Les variables des paramètres

❖ Comme toutes les commandes, vos scripts bash peuvent eux aussi accepter des paramètres.

❖ Récupération des paramètres:

`$#` : contient le nombre de paramètres

`$0` : contient le nom du script exécuté

`$1` : contient le premier paramètre

`$2` : contient le second paramètre

... ;

Exemple:

```
#!/bin/bash
```

```
echo "Vous avez lancé $0, il y a $# paramètres"
```

```
echo "Le paramètre 1 est $1"
```

```
./test.sh param1 param2 param3
```

Vous avez lancé ./test.sh, il y a 3 paramètres Le paramètre 1 est param1

Les Variables

Les tableaux

Le bash gère également les variables « tableaux ». Ce sont des variables qui contiennent plusieurs cases:

Exemples:

- `tableau=('valeur0' 'valeur1' 'valeur2')`
`${tableau[2]}`
- `#!/bin/bash`
`tableau=('valeur0' 'valeur1' 'valeur2')`
`tableau[5]='valeur5'`
`echo ${tableau[*]}`

Les structures alternatives

Instruction if

Syntaxe 1 :

```
if [ condition ]  
then  
action1  
fi
```

Syntaxe 2 :

```
if [ condition ]  
then  
    action1  
else  
    action2  
fi
```

- ❖ Teste de la valeur d'une variable
- ❖ Tester les valeurs des paramètres
- ❖ Comparaison de deux variables

Les structures alternatives

Instructions if imbriquées

```
if [ condition 1 ]  
then  
action1  
elif [ condition 2 ]  
then  
action2  
elif [ condition 3 ]  
action3  
...  
else  
actionN  
Fi
```

❖ Inversement du test: if [!condition]

Les structures alternatives

Instructions case

```
case valeur_de_variable in
val1)
commandes
;;
val2)
commandes
;;
...
*)
commandes
esac
```

Les structures alternatives

Les types de tests:

- ❖ Il est possible d'effectuer trois types de tests différents en bash :
 - Tests des chaînes de caractères
 - Tests des nombres
 - Tests des fichiers

❖ Tests des chaînes de caractères

Condition	Signification
<code>\$chaine1 = \$chaine2</code>	Vérifie si les deux chaînes sont identiques. Il est aussi possible d'écrire « == » pour les habitués du langage C.
<code>\$chaine1 != \$chaine2</code>	Vérifie si les deux chaînes sont différentes.
<code>-z \$chaine</code>	Vérifie si la chaîne est vide.
<code>-n \$chaine</code>	Vérifie si la chaîne est non vide.

Les structures alternatives

Tests des nombres

Condition	Signification
<code>\$num1 -eq \$num2</code>	Vérifie si les nombres sont égaux (<i>equal</i>). À ne pas confondre avec le « = » qui, lui, compare deux chaînes de caractères.
<code>\$num1 -ne \$num2</code>	Vérifie si les nombres sont différents (<i>not equal</i>). Encore une fois, ne confondez pas avec « != » qui est censé être utilisé sur des chaînes de caractères.
<code>\$num1 -lt \$num2</code>	Vérifie si <code>num1</code> est inférieur (<) à <code>num2</code> (<i>lower than</i>).
<code>\$num1 -le \$num2</code>	Vérifie si <code>num1</code> est inférieur ou égal (<=) à <code>num2</code> (<i>lower or equal</i>).
<code>\$num1 -gt \$num2</code>	Vérifie si <code>num1</code> est supérieur (>) à <code>num2</code> (<i>greater than</i>).
<code>\$num1 -ge \$num2</code>	Vérifie si <code>num1</code> est supérieur ou égal (>=) à <code>num2</code> (<i>greater or equal</i>).

Les structures alternatives

Tests des fichiers

Condition	Signification
-e \$nomfichier	Vérifie si le fichier existe.
-d \$nomfichier	Vérifie si le fichier est un répertoire.
-f \$nomfichier	Vérifie si le fichier est un... fichier. Un vrai fichier cette fois, pas un dossier.
-L \$nomfichier	Vérifie si le fichier est un lien symbolique
-r \$nomfichier	Vérifie si le fichier est lisible (r).
-w \$nomfichier	Vérifie si le fichier est modifiable (w).
-x \$nomfichier	Vérifie si le fichier est exécutable (x).
\$fichier1 -nt \$fichier2	Vérifie si fichier1 est plus récent que fichier2 (<i>newer than</i>).
\$fichier1 -ot \$fichier2	Vérifie si fichier1 est plus vieux que fichier2 (<i>older than</i>).

Les structures alternatives

Exercice:

Ecrire un script qui demande à l'utilisateur d'entrer le nom d'un répertoire et qui vérifie si c'en est bien un.

```
#!/bin/bash
read -p 'Entrez un répertoire : ' repertoire
if [ -d $repertoire ]
then
    echo 'Bien, vous avez entré un répertoire'
else
    echo 'ce n'est pas un répertoire...'
fi
```

Les structures alternatives

❖ On peut effectuer plusieurs tests à la fois en utilisant les opérateurs logiques:

➤ **&&** : signifie « et »

➤ **||** : signifie « ou »

Exemple: **if** [condition1] **&&** [condition2]

Exercice:

Ecrire un script shell qui nous informe:

si le fichier **accepte au moins** un paramètre **et** si le premier paramètre vaut MID2017 **afficher** le message ' vous avez trouvez le mot de passe' si non afficher 'mot de passe erroné'.

```
#!/bin/bash
```

```
if [ $# -ge 1 ] && [ $1 = 'MID2017' ]
```

```
then
```

```
    echo "msg1"
```

```
else
```

```
    echo "msg2"
```

```
fi
```


TP4 : Programmation Shell

Exercice 1 : Tapez et exécuter les scripts shell suivants

Script1.sh

```
#!/bin/sh  
echo "Bonjour.SUSER"
```

script2.sh

```
#!/bin/sh  
Sudo aptitude update  
Sudo aptitude dist-upgrade
```

Script3.sh

```
#!/bin/sh  
Echo -n "entrez un nom de fichier"  
Read file  
If [ -e "$file" ] ;then  
Echo " le fichier exist"  
Else  
Echo " le fichier n'existe pas"  
Fi
```

Exercice 2 :

Ecrire un script shell qui nous informe si le fichier accepte au moins un paramètre et si le premier paramètre vaut DWAM2018 afficher le message 'vous avez trouvé le mot de passe' si non afficher 'mot de passe erroné'.

Exercice 3 :

Créer un script qui demande à l'utilisateur de saisir une note et qui affiche un message en fonction de cette note :

- "très bien" si la note est entre 16 et 20
- "bien" lorsqu'elle est entre 14 et 16
- "assez bien" si la note est entre 12 et 14
- "moyen" si la note est entre 10 et 12
- "insuffisant" si la note est inférieure à 10

Les structures répétitives

Instruction for

```
for var in liste
do
commandes
done
```

Exemple :

```
for file in *.sh
do
cat $file
Done
```

```
for i in `seq 1 10`
do
echo $i
done
```

```
for fichier in `ls`
do
mv $fichier $fichier-old
done
```

Les structures répétitives

Tests des nombres

```
while [ condition ]  
do  
commandes  
Done
```

Exemple:

1) Que fait le script suivant?

```
#!/bin/bash
```

```
while [ -z $reponse ] || [ $reponse != 'DWAM2018' ]  
do  
    read -p 'Ecrire DWAM2018 : ' reponse  
done
```

Les Fonctions

Dans un programme script, on peut regrouper un ensemble d'instructions dans une fonction qui sera identifiée par un nom.

```
# déclaration méthode 1
maFonction ()
{
  bloc d'instructions
}
#appel de ma fonction
maFonction
```

```
# déclaration méthode 2
function maFonction
{
  bloc d'instructions
}
#appel de la fonction
    maFonction
```

Exercices

Exercices:

1) La boucle while

Ecrire un programme shell qui affiche les arguments du programme, dans l'ordre d'apparition (1er argument en premier). Si le programme n'a aucun argument, afficher « *sans argument* ».

Indication : Utilisez la commande shift. (décalage des paramètres)

2) La boucle for

Ecrire un programme shell qui affiche tous les sous-répertoires du répertoire courant, *en utilisant une boucle*.

Merci de votre
attention

Solutions:

Exercice 1

```
#!/bin/sh
if [ $# -eq 0 ]; then
echo "sans argument"
exit 0
fi
while [ $# -gt 0 ]; do
echo $1
shift
done
```

Exercice 2:

```
#!/bin/sh
for rep in *; do          # pour chaque fichier "rep"
                           du repertoire courant
if [ -d "$rep" ]; then    # si rep est un repertoire
echo $rep                 # affiche rep
fi
done
```